

33. App Discovery

The App Discovery feature enables accessories to retrieve a list of installed apps on the device capable of communicating with the accessory. See [App Launch](#) (page 299) to make use of the list.

33.1 Requirements

Accessories supporting the App Discovery feature using iAP2 shall send or receive the following iAP2 control session message(s):

[StartAppDiscoveryUpdates](#) (page 880)

[AppDiscoveryUpdate](#) (page 881)

Accessories supporting the App Discovery feature using iAP2 may also send or receive the following iAP2 control session message(s):

[StopAppDiscoveryUpdates](#) (page 882)

33.2 Usage

A [StartAppDiscoveryUpdates](#) (page 880) message from the accessory to the device starts the generation of [AppDiscoveryUpdate](#) (page 881) messages from the device.

[AppDiscoveryUpdate](#) (page 881) messages are sent after the App Discovery feature starts and contain a list of matching apps. A message containing the `AppListAvailable` and `AppListCount` parameters may be sent separately from one containing the `AppList` parameters.

Additional [AppDiscoveryUpdate](#) (page 881) messages will be sent if the app list or any parameters change. Accessories shall assume the last reported values remain constant until they are overwritten by a subsequent [AppDiscoveryUpdate](#) (page 881) message.

34. App Launch

Accessories supporting the App Launch feature can request a device launch an app on its behalf.

Figure 34-1 App Launch Alert



This feature is supported on tvOS 10.2 or later.

34.1 Requirements

Accessories supporting the App Launch feature using iAP2 shall send or receive the following iAP2 control session message(s):

[RequestAppLaunch](#) (page 880)

App Launch messages shall only be sent in response to a [Direct User Action](#) (page 52).

The launched app shall communicate with the accessory using the [External Accessory Protocol](#) (page 352).

34.2 Usage

If a device does not support the app launch feature, the device will reject all accessory identification attempts including the [RequestAppLaunch](#) (page 880) message.

Accessories shall not assume the app has launched, because the request may be denied by the device or the user. If positive confirmation of an app launch is needed, the app shall open an [External Accessory Protocol](#) (page 352) session with the accessory.

By default, [RequestAppLaunch](#) (page 880) will alert the user and ask for confirmation before launching the app to the foreground. This behavior is appropriate for apps that do not communicate with an accessory when backgrounded.

If the app can communicate with an accessory while backgrounded, accessories should set the `AppLaunchMethod` parameter to 1 (launch without alert), which will not display a user alert asking for confirmation. Additionally, [RequestAppLaunch](#) (page 880) will attempt to launch the app into the background if:

- The device is locked.
- The device is unlocked but some other app is already in the foreground.

Accessories shall not attempt to influence the outcome of an App Launch request. Examples include but are not limited to emulating user actions using HID (which is already prohibited) or other accessory interface features.

34.3 Test Procedures

34.3.1 iAP2 Tests

1. Verify the [RequestAppLaunch](#) (page 880) message is only sent after a direct user action.
2. If the accessory implements *Launch with user alert*:
 - a. Verify the app is not launched when 'Ignore' is selected when the device prompts the user to Allow.
3. If the accessory implements *Launch without user alert*:
 - a. Verify the app is launched without alert after a direct user action.

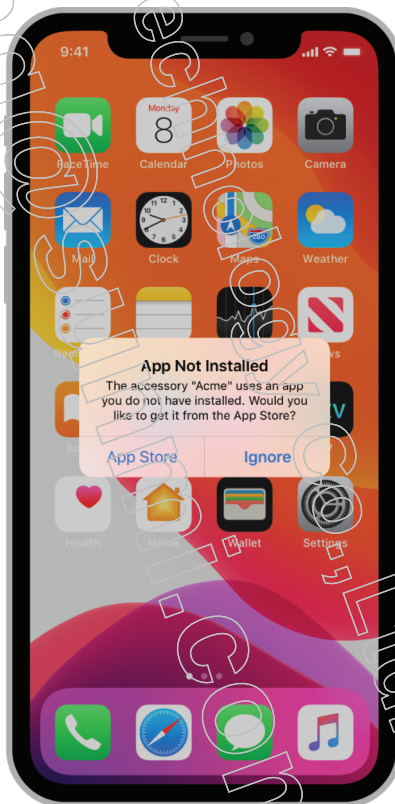
35. App Match

The App Match feature enables accessories supporting the [External Accessory Protocol](#) (page 352) feature to match with compatible apps on the App Store.

When connected for the first time, the device asks the user if they would like to visit the App Store and view compatible apps. Subsequently, this action may be repeated by the user using *Settings > General > About > 'Accessory Name' > 'Find App for this Accessory'*.

Matched apps are listed in alphabetical order with one exception. If the accessory works with apps from multiple development teams/companies, the accessory may provide a preferred Team ID to place apps from the preferred team at the top of the list.

Figure App Match Alert
35-1



35.1 Requirements

To support App Match during [Accessory Identification](#) (page 281), the accessory:

- Shall declare at least one [External Accessory Protocol](#) (page 352) matching one or more apps allowed to declare support for the same External Accessory Protocol(s).
- May additionally declare a preferred Team ID matching one or more apps from a single app development team.

35.1.1 External Accessory Protocol

Unless otherwise specified, accessories shall fully implement at least one [External Accessory Protocol](#) (page 352) to communicate with iOS apps supporting App Match.

Accessories may use the App Match feature with more than one External Accessory Protocol. If multiple protocols are listed, the App Store will display a list of apps supporting one or more compatible protocols.

35.1.2 Team ID

Team ID used for App Match shall belong to the accessory developer.

A maximum of one Team ID can be declared for App Match.

35.2 Usage

35.2.1 External Accessory Protocol

Accessories shall support App Match for a specific External Accessory Protocol by setting the [MatchAction](#) (page 874) parameter in the [ExternalAccessoryProtocol](#) (page 874) parameter group to an appropriate value during accessory identification.

35.2.2 Team ID

Accessories may declare app compatibility by providing a `AppMatchTeamID` parameter in the [IdentificationInformation](#) (page 872) message. The parameter's value shall be a valid Apple Developer Team ID.

Team ID is supplied by Apple and unique to a specific development team. Team ID can be found in Apple Developer Account details under Membership (see <https://developer.apple.com/account/>).